

Speaker Script — Topic 12: Deployment

Next.js Fundamentals Series | Session 12 of 13 Estimated Duration: 55–70 minutes

SLIDE 1 — Title Slide

"Welcome to Topic 12 — Deployment.

This is one of the most exciting sessions in the entire course, because today is the day your app stops living on your laptop and starts living on the internet — for real, accessible to anyone, anywhere in the world.

We've covered a huge amount of ground to get here: routing, components, styling, data fetching, API routes, forms, authentication, SEO. Today is where all of that comes together. We're going to take a Next.js project and ship it to production.

By the end of today you will have deployed a live app on Vercel with your own URL. Not a mock. Not a demo. An actual live website.

Let's go."

SLIDE 2 — Agenda

"Here's everything we're covering today — seven topics.

We start with **how Next.js actually builds your app** — because understanding the build output changes how you write code. Then we look at testing that build locally before you push anywhere.

After that: **environment variables** — this is where most beginners make security mistakes, so we'll spend time on this properly. Then the main event: **deploying to Vercel**, step by step.

We'll also cover **custom domains and HTTPS**, the **CI/CD pipeline** that makes deployment automatic, and finally a **post-deploy checklist** — because going live isn't the same as being production-ready.

A lot of ground to cover. Let's start with the build process."

SLIDE 3 — How Next.js Builds

"When you run `npm run build`, something really smart happens.

Next.js looks at every single page in your app and asks: *what's the best way to serve this page?* It has three options.

The circle symbol (○) means Static — SSG. The page is pre-rendered at build time. HTML is generated once, stored as a file, and served instantly to every user with zero processing. No server needed. This is the fastest possible way to deliver a web page. Your `/about` page, your `/contact` page, your homepage — anything that doesn't need real-time data — these should all be Static.

The lambda symbol (λ) means Server — SSR. This page is rendered fresh on every single request. The server runs your code, fetches data, and builds the HTML each time. This is slightly slower but the data is always 100% current. Use this for dashboards, user profiles, personalised feeds — anything where each user sees different data.

The half-circle symbol (◐) means ISR — Incremental Static Regeneration. This is the clever middle ground. The page is generated statically at build time, but Next.js will regenerate it in the background after a set interval — maybe every 60 seconds. So users always get a fast static page, but the content stays fresh. Perfect for e-commerce product pages, news articles, blog posts.

Now look at the build output at the bottom of the slide. This is what you'll actually see in your terminal after running `npm run build`. Each route has a symbol showing which strategy was chosen. You can see the size of each page and the total JavaScript sent to the browser.

Two things to watch for in this output: First — if a page you expected to be static shows as dynamic (λ), that means you accidentally made it dynamic somewhere. Find the `fetch()` call without caching and fix it. Second — if any page's 'First Load JS' is very large (over 200kB), you have a performance problem that needs to be addressed before you go live."

SLIDE 4 — The next start Command

"Before you push to Vercel, there's a critical step most beginners skip: testing the production build locally.

Here's the thing — your development server (`npm run dev`) behaves very differently from production. In dev mode, errors show you full stack traces, hot reload is running, and nothing is optimised. In production, Next.js removes all of that, minifies your JavaScript, and runs much more strictly.

So there's a category of bugs that only appear in production mode. If you've ever pushed a deployment that looked fine locally but immediately broke on Vercel — this is exactly why.

The workflow is simple. Two commands. Run `npm run build` first. This generates the production output. If this command throws an error, your deployment would have failed on

Vercel too — better to catch it now. Then run `npm start`. This starts the production server on your localhost.

Now check your app in the browser exactly the same way you would check it live. The checklist on the right tells you what to test:

- Every page should load without a 500 error
- All images, fonts, and assets should appear correctly
- API routes should respond properly
- Environment variables should be read correctly — this catches misconfigured `.env` files before they cause a production incident
- Navigation and redirects should work

If anything fails at this stage, you fix it before you push. You've just saved yourself from a broken deployment in front of real users."

SLIDE 5 — Environment Variables

"Let's talk about environment variables — and why getting this wrong is a security risk, not just a bug.

The core idea is simple: you have configuration values that change between environments — database connection strings, API keys, JWT secrets. You never hardcode these in your source code, because your source code lives in GitHub and could be seen by others. Instead, you put them in special files that are loaded at runtime.

Let's walk through the table on the left.

`.env` — loaded in all environments. Commit this to git only if it contains no secrets. Use it for default values.

`.env.local` — loaded only during local development with `npm run dev`. **Never commit this file.** This is where your real database URL lives, your real API keys, your JWT secret. Add `.env.local` to your `.gitignore` file.

`.env.development` — loaded only in dev mode. Safe to commit if it contains no secrets.

`.env.production` — loaded only during `npm run build` and `npm start`. **Never commit this.** Add it to `.gitignore`.

`.env.test` — for testing only.

Now look at the code on the right. In your API routes and server-side code, you access variables with `process.env.VARIABLE_NAME`. They're only available on the server — the browser never

sees them.

The critical rule you must remember: any variable prefixed with `NEXT_PUBLIC_` is bundled into your JavaScript and sent to the browser. This is intentional — it's how you pass configuration like your site URL to client-side code. But it means **you must NEVER prefix a secret with `NEXT_PUBLIC_`**. Your database password with `NEXT_PUBLIC_DATABASE_URL` would be visible to every user who opens DevTools. This is a serious security vulnerability.

Look at the warning card in red — please read this carefully.

On Vercel, you add your environment variables in the project dashboard under Settings → Environment Variables. Vercel injects them at build time and runtime. You can set different values for Production, Preview, and Development environments."

SLIDE 6 — Deploying to Vercel

"Now the main event. Six steps to take your Next.js app from your laptop to the internet.

Step 1 — Push to GitHub. If you haven't already, create a GitHub repository and push your code. These are the three commands on the card: `git add .`, `git commit`, `git push`. Your code is now on GitHub.

Step 2 — Import on Vercel. Go to vercel.com, sign up or log in with your GitHub account, click 'New Project', and import your repository. Vercel will ask GitHub for permission to read your repos — grant it.

Step 3 — Configure Project. Vercel will automatically detect that you're using Next.js. The build command (`npm run build`), output directory (`.next`), and install command (`npm install`) are all set automatically. You usually don't touch anything here.

Step 4 — Add Environment Variables. Before you click deploy, add your environment variables in the configuration panel. This is your `DATABASE_URL`, your `JWT_SECRET`, anything from your `.env.local` file. Vercel will inject these into the build.

Step 5 — Click Deploy. Vercel runs `npm run build` on its servers, optimises all your assets, and distributes the output to its global edge network. This takes 30 seconds to 2 minutes depending on your project.

Step 6 — Your app is live. You get a URL like `yourapp.vercel.app`. Visit it. Your app is live on the internet.

The thing I want you to appreciate about this workflow: **Vercel runs the same `npm run build` command you run locally.** There are no deployment scripts, no server configuration, no Docker containers. You write code, push to GitHub, and Vercel handles everything else.

And here's the best part — you only do this setup once. After today, every time you `git push` to main, Vercel automatically redeploys your app. CI/CD out of the box, zero configuration."

SLIDE 7 — Custom Domains & HTTPS

"Your app is live at `yourapp.vercel.app` — that's great for sharing. But for a real product, you want your own domain like `myapp.com`. Let's walk through that.

Seven steps, and you can see them on the left.

Go to your Vercel project, click Settings, then Domains, then Add. Type your domain. Vercel will show you the exact DNS records you need to create — usually an A record for the root domain and a CNAME for the www version.

Now go to wherever you bought your domain — GoDaddy, Namecheap, Google Domains, Cloudflare — and create those DNS records exactly as Vercel specifies. The table on the right shows the typical values. An A record pointing `@` to Vercel's IP address, a CNAME pointing `www` to `cname.vercel-dns.com`.

Then wait. DNS propagation takes anywhere from 5 minutes to a few hours depending on your registrar and your old TTL settings. You can track progress on dnschecker.org.

Two things I want to highlight in the info cards:

HTTPS is completely automatic. Vercel provisions SSL certificates via Let's Encrypt. You don't pay for it, you don't set it up, you don't renew it. Your site gets a green padlock and HTTPS from day one. This used to cost hundreds of dollars a year and require significant technical setup. Now it's zero effort.

And your site is served from Vercel's global CDN — over 100 edge locations worldwide. When a user in Mumbai visits your site, they're served by a Vercel server in Mumbai, not one in the US. This makes your site fast everywhere on earth, not just near your server."

SLIDE 8 — CI/CD & Preview Deployments

"This is where Vercel goes from good to genuinely magical.

Look at the flow diagram at the top. You write code locally. You push to a branch on GitHub. Vercel detects the push and automatically starts a build. Within 60-90 seconds, you have a unique preview URL. You share that with your team or client. They review it. You merge the branch to main. Vercel deploys to production. Automatically.

Let me explain the three types of deployment on the left panels.

Production Deployments happen when you push to your main branch. This updates your live site — `yourapp.vercel.app` and your custom domain. Vercel keeps every previous production deployment. If the new version has a bug, you can roll back to yesterday's version in one click, with zero downtime.

Preview Deployments happen on every other push. Every single branch. If you create a branch called `feature/new-navbar` and push it, Vercel gives it a URL like `yourapp-git-feature-new-navbar-you.vercel.app`. This URL is live immediately. You can share it with a client before you merge. You can open it on your phone and test it. Your whole team can review it.

Think about what this means in practice. A client asks 'Can I see the new design before it goes live?' In the old world, you'd need a staging server. In the Vercel world, you share the preview URL from five minutes ago.

Rollbacks and monitoring are in the third panel. If a deployment breaks something, you go to the Vercel dashboard, find the previous good deployment, click 'Promote to Production', and your site is back to the old version — instantly. No git revert, no emergency commits. One click.

You also get real-time function logs — so if your API route throws an error in production, you can see the `console.log` output in the Vercel dashboard. And Vercel Analytics shows you Core Web Vitals — real performance data from real users."

SLIDE 9 — Post-Deploy Checklist

"Deploying your app is not the same as being production-ready. There's a checklist of things you need to verify after your first deployment — and I'd encourage you to print this slide or save it.

Performance. Run a Lighthouse audit. Open Chrome DevTools, go to the Lighthouse tab, and run an audit on your live URL. You should score 90 or above on Performance. Look at Core Web Vitals — specifically LCP (Largest Contentful Paint), FID (First Input Delay), and CLS (Cumulative Layout Shift). Check that you're using `next/image` for all images — not raw `<img>` tags. And look at the bundle analyser to check you're not shipping massive JavaScript files.

SEO. Every page should have a unique, descriptive title and meta description — we covered this in Topic 11. Open Graph tags should work — paste your URL into `opengraph.xyz` to see how it looks when shared on LinkedIn or WhatsApp. You should have a `sitemap.xml` so search engines can crawl your pages, and a `robots.txt` telling them what to index.

Security. Check that no secrets ended up in your client-side bundle — search your build output for anything that looks like a password or API key. Verify your API routes validate input before processing it. Make sure HTTPS is working — no mixed-content warnings in the browser console. Consider adding security headers like X-Frame-Options and Content-Security-Policy.

Error Monitoring. Set up Sentry or a similar service — it takes about 10 minutes and it's free for small projects. Sentry will email you every time an unhandled exception occurs in your app, with a full stack trace. Make sure you have custom 404 and 500 error pages (`app/not-found.jsx`) and (`app/error.jsx`). And before you call the launch done, manually click through every important user flow — sign up, log in, create something, delete something.

I'll be honest — most developers skip half of this checklist on their first deployment. Then they wonder why their site is slow, or why their error page shows a raw Next.js 500 page to users. Do this properly once and you'll never forget it."

SLIDE 10 — Cheatsheet

"This is your deployment reference — everything in one slide. Save it, screenshot it, or keep the PDF.

The commands are the most important column. `npm run build` — you'll run this constantly. `npm start` — run this before every deployment to catch production bugs early. Push to GitHub, not directly to Vercel. Push to a branch for preview, merge to main for production.

The environment variable rows are the ones I want you to double-check right now — in your own project. Is your `.env.local` in your `.gitignore`? Are you using `NEXT_PUBLIC_` correctly — only for values that are genuinely safe to expose?

The build output types row at the bottom: `Static`, `Server`, `ISR`. When you run your next build, look at that output. If anything is dynamic that should be static, that's a performance issue to fix. If anything is static that should be dynamic, your users might see stale data.

Any questions on the cheatsheet before I move to the summary?"

(Pause for questions)

SLIDE 11 — Key Takeaways

"Four things to lock in from today.

One — `npm run build` analyses every page and outputs the optimal format. Static, Server, or ISR — Next.js decides automatically based on how you wrote the page. Reading the build output log is a skill. You should check it every time you build.

Two — environment variables are how you keep secrets out of your code. Use `.env.local` for local development. Add the same keys in Vercel's dashboard for production. Only variables prefixed with `NEXT_PUBLIC_` are safe to expose to the browser — never prefix a secret with that.

Three — Vercel deploys automatically on every git push, zero server management. Connect your repository once. Push to main, your site updates. Push to any branch, you get a preview URL. Rollback any deployment with one click. You never SSH into a server again.

Four — going live is not the finish line. Run Lighthouse. Set up error monitoring. Check your SEO tags. Verify security. Test every user flow. Production is the start of maintenance, not the end of building.

You now have everything you need to ship real Next.js applications. The next session — Topic 13 — is the Final Project. You'll apply every concept from this course to build and deploy a complete app from scratch."

SLIDE 12 — Closing Slide

"That's Topic 12 done — and you are now a developer who can build AND ship.

Before Topic 13, I want you to complete five tasks — these are not optional, they're the practice that makes the final project work.

One — run `npm run build` on your existing project right now. Read the output. Note which pages are static, which are server, which are ISR. Fix any build errors before you proceed.

Two — push your project to a GitHub repository if it isn't there already.

Three — deploy to Vercel. Go to vercel.com, connect the repo, add your environment variables, click Deploy. The whole process takes under 5 minutes.

Four — add your environment variables in the Vercel dashboard. Check that your app actually works — that the database connects, that authentication works, that API routes respond.

Five — share your live Vercel URL. Send it to a friend, a family member, or post it. Your app is on the internet. That's something to be proud of.

Coming up in Topic 13 — the Final Project. We'll plan, build, and deploy a complete full-stack Next.js application from zero to production. Everything you've learned over these 12 sessions comes together.

See you there — and well done for making it this far."

Teaching Notes & Timing

Slide	Topic	Time	Key Focus
1	Title	2 min	High energy — this is the exciting session

Slide	Topic	Time	Key Focus
2	Agenda	2 min	Overview only, don't go deep
3	Build Output	10 min	Slow down on <code>λ</code> — demo a real build in terminal
4	Local Prod Testing	7 min	Live demo: run <code>npm run build && npm start</code>
5	Env Variables	10 min	Critical — go through the <code>NEXT_PUBLIC_</code> rule twice
6	Vercel Deploy	12 min	Live demo if possible — deploy a real app
7	Custom Domains	6 min	Show Vercel's domain dashboard
8	CI/CD & Previews	7 min	Demo: push a branch, show the preview URL
9	Post-Deploy Checklist	6 min	Run Lighthouse live on a deployed app
10	Cheatsheet	3 min	Student Q&A
11	Summary	3 min	Slow recap
12	Closing	2 min	Assign 5 tasks clearly — they need to deploy today

Detailed Concept Explanations

What is SSG (Static Site Generation)?

SSG means the HTML for your page is created at **build time** — before any user visits it. When you run `npm run build`, Next.js renders the page to HTML and saves it as a file. When a user visits that URL, the server just sends the file. No code runs. No database is queried. It's as fast as serving a text file.

When to use SSG: Any page where the content doesn't change based on who's viewing it and doesn't need to be updated more than once per build. Marketing pages, about pages, documentation, blog posts that don't change often.

The limitation: If the content changes, users won't see the new version until you rebuild and redeploy. For content that changes daily or less, this is fine. For content that changes by the minute, use SSR or ISR.

What is SSR (Server-Side Rendering)?

SSR means the HTML is generated **on every request**. When a user visits the URL, Next.js runs your page's code on the server, fetches whatever data it needs, renders the HTML, and sends it to

the browser. This happens fresh every single time.

When to use SSR: When the page content is different for each user (dashboards, profiles, authenticated pages), or when the data changes so frequently that ISR isn't fresh enough.

The cost: Each request requires server processing. This is slightly slower than SSG and requires a running Node.js server (not just static file hosting).

What is ISR (Incremental Static Regeneration)?

ISR is the clever middle ground. The page is initially generated statically (like SSG). But after a set time interval — say 60 seconds — when the next user visits, Next.js serves the stale cached version **and simultaneously** regenerates the page in the background. The next request after that gets the fresh version.

```
js

// In your page component or fetch call:
fetch('/api/products', { next: { revalidate: 60 } })
// This page re-generates every 60 seconds at most
```

When to use ISR: Product pages, news articles, blog posts, anything that changes occasionally but not constantly. Users always get a fast cached page, and the content stays reasonably fresh.

What is Vercel?

Vercel is a cloud deployment platform built by the same team that created Next.js. It's specifically optimised for Next.js apps. When you deploy to Vercel:

- Your code is built using `npm run build`
- Static pages (○) are stored on a global CDN — served from the closest location to each user
- Server pages (λ) run as serverless functions — they only execute when a request arrives, costing nothing when idle
- ISR pages (●) are cached at the CDN with background regeneration

This architecture is why Next.js apps on Vercel are so fast. Static content is served from a CDN (milliseconds). Server content runs as serverless functions (no idle server costs). The whole thing scales automatically.

What is CI/CD?

CI stands for Continuous Integration — automatically building and testing code every time it's pushed to the repository. **CD** stands for Continuous Deployment — automatically deploying code to production after it passes the build process.

Vercel gives you CD out of the box. Every push to your main branch triggers a new production deployment. Every push to any other branch creates a preview deployment. You don't configure any of this — it just works.

More advanced teams add CI (automated tests) before the deployment step — using GitHub Actions to run `npm test` or Playwright tests before Vercel deploys. This ensures code that breaks tests never reaches production.

Environment Variable Security Explained

There are two places your Next.js code runs: the **server** and the **browser**.

Server-side code (API routes, Server Components, `getServerSideProps`) runs in Node.js. It has access to `process.env.ANYTHING` — including secrets. This code is never sent to the user.

Browser-side code (Client Components, any JavaScript that runs in the browser) is **sent to the user as a downloaded file**. Anyone can open DevTools and read it. If you accidentally include a database password in client-side code, every visitor to your site can steal it.

`NEXT_PUBLIC` is Next.js's way of saying: "I know this variable is safe to include in the browser bundle." You use it for things like your public site URL, Google Analytics ID, Stripe publishable key (which is meant to be public), etc.

Everything WITHOUT `NEXT_PUBLIC` stays server-side only. `DATABASE_URL`, `JWT_SECRET`, private API keys — these never reach the browser.

Common Student Mistakes

Mistake 1: Committing `.env.local` to git Add `.env.local` and `.env.production` to your `.gitignore` before your first commit. If you've already committed them, remove them from git history (`git rm --cached .env.local`) and rotate all your secrets immediately.

Mistake 2: Using `NEXT_PUBLIC` for secrets `NEXT_PUBLIC_DATABASE_URL` will put your database connection string in the JavaScript bundle. Every user's browser will download it. Never do this.

Mistake 3: Not testing the production build locally Always run `npm run build && npm start` before pushing. Catch build errors locally, not on Vercel.

Mistake 4: Not adding env variables in Vercel You add variables to `.env.local` locally, but you must also add them in Vercel's dashboard. They're separate. A common mistake is deploying and then wondering why the database connection fails — the env vars are missing on Vercel.

Mistake 5: Deploying without running the post-deploy checklist Lighthouse, SEO tags, error monitoring — these feel optional until your first production incident or your first time someone

shares your page on LinkedIn and the preview image is wrong.

Script prepared for Next.js Fundamentals Series — Topic 12: Deployment